ARP Spoofing Tool using Python

Roll Number: 03 | Platform: Kali Linux | Language: Python | Library: Scapy

1. Project Title

ARP Spoofing Tool using Python (Roll No: 03)

2. Objective

The objective of this project is to implement an ARP (Address Resolution Protocol) Spoofing tool using Python and the Scapy library. The tool performs ARP cache poisoning to intercept network traffic between a victim machine and the gateway, demonstrating a Man-in-the-Middle (MITM) attack in an isolated lab environment.

3. Software Requirements

Component	Details
OS	Kali Linux (Attacker) + Metasploitable2 (Victim)
Platform	Oracle VirtualBox — Host-Only Adapter
Language	Python 3
Library	Scapy 2.6.1
Network	Subnet: 10.16.113.0/24
Attacker IP	Kali Linux — 10.16.113.62
Victim IP	Metasploitable2 — 10.16.113.57
Gateway IP	10.16.113.196

4. Step 1 — Lab Setup: Check Network Configuration

The lab was set up using VirtualBox with Kali Linux as the attacker and Metasploitable2 as the victim. Both VMs were configured on a Host-Only Adapter to ensure network isolation. The following command was run on Metasploitable2 to verify the network configuration:

Command on Metasploitable2: `ifconfig`

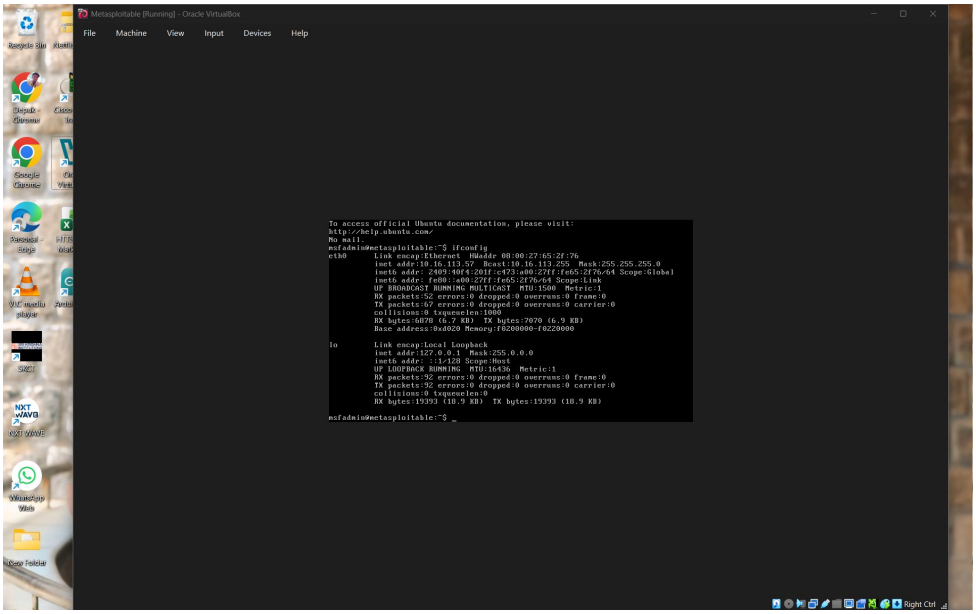


Figure 1: Metasploitable2 network configuration — eth0 IP: 10.16.113.57, MAC: 08:00:27:65:2f:76

5. Step 2 — Capture ARP Table Before Poisoning (TC-1)

Before running the ARP spoofing tool, the ARP table on Metasploitable2 was recorded. The victim was first made to communicate with the gateway using ping, then the ARP table was checked to confirm the real gateway MAC address:

Commands on Metasploitable2:

```
ping -c 3 10.16.113.62
```

```
arp -n
```

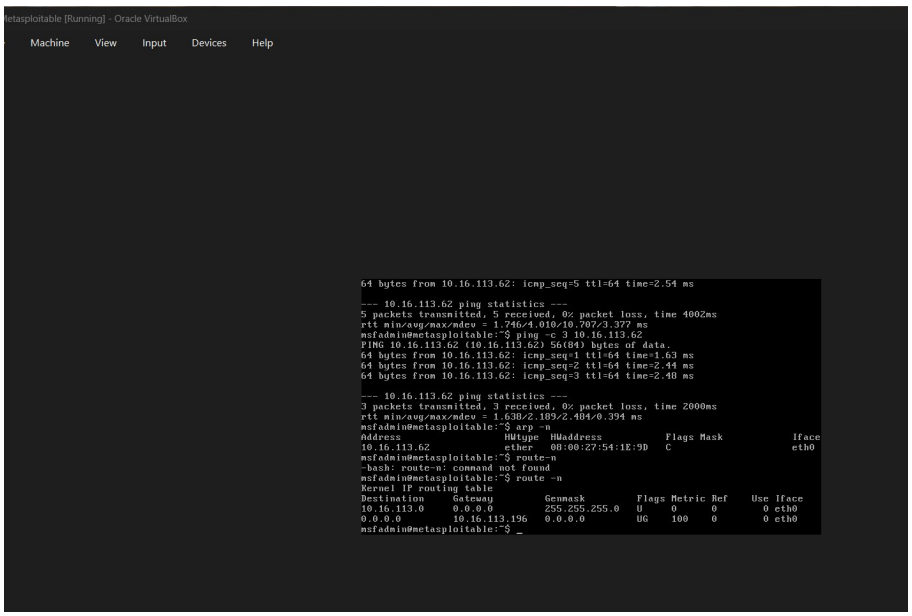
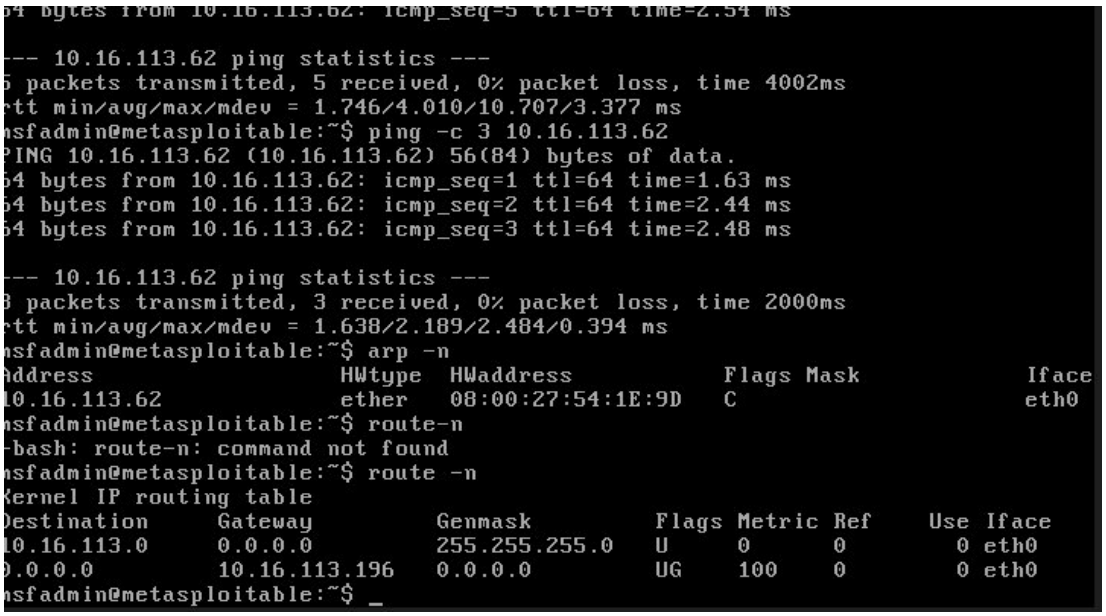


Figure 2: ARP table BEFORE poisoning — gateway 10.16.113.62 shows real MAC 08:00:27:54:1E:9D

6. Step 3 — Verify Gateway via Routing Table

The routing table was verified on Metasploitable2 to confirm the default gateway. This confirms the gateway IP is 10.16.113.196 which is the target for ARP poisoning:

Command: `route -n`



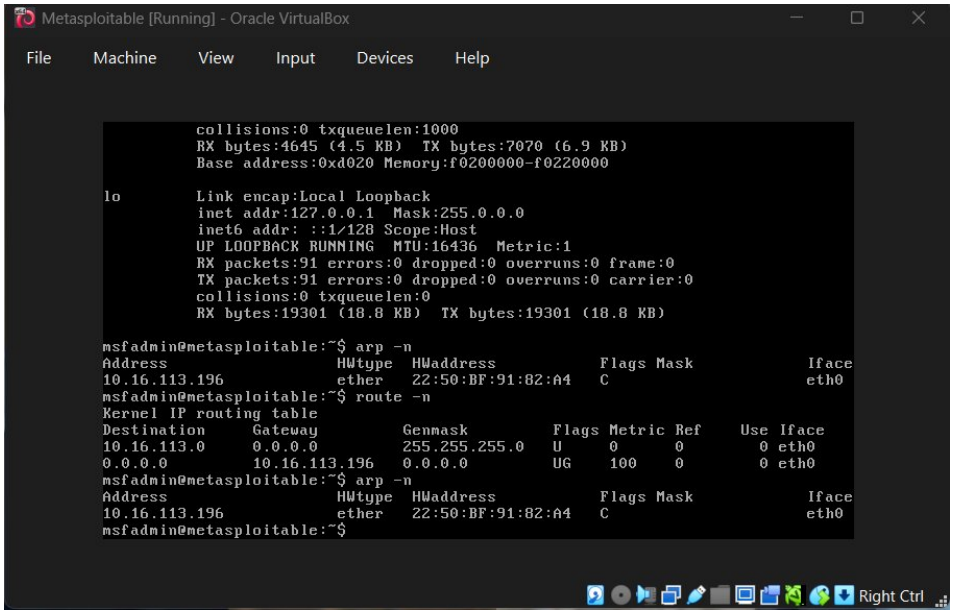
```
64 bytes from 10.16.113.62: icmp_seq=5 ttl=64 time=2.54 ms

--- 10.16.113.62 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4002ms
rtt min/avg/max/mdev = 1.746/4.010/10.707/3.377 ms
msfadmin@metasploitable:~$ ping -c 3 10.16.113.62
PING 10.16.113.62 (10.16.113.62) 56(84) bytes of data.
64 bytes from 10.16.113.62: icmp_seq=1 ttl=64 time=1.63 ms
64 bytes from 10.16.113.62: icmp_seq=2 ttl=64 time=2.44 ms
64 bytes from 10.16.113.62: icmp_seq=3 ttl=64 time=2.48 ms

--- 10.16.113.62 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2000ms
rtt min/avg/max/mdev = 1.638/2.189/2.484/0.394 ms
msfadmin@metasploitable:~$ arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
10.16.113.62             ether   08:00:27:54:1E:9D   C                     eth0
msfadmin@metasploitable:~$ route-n
-bash: route-n: command not found
msfadmin@metasploitable:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
10.16.113.0     0.0.0.0         255.255.255.0   U     0      0        0 eth0
0.0.0.0         10.16.113.196   0.0.0.0         UG    100    0        0 eth0
msfadmin@metasploitable:~$ _
```

Figure 3: Routing table showing default gateway 10.16.113.196 via eth0

7. Step 4 — ARP Table Showing Gateway MAC (Pre-Attack)

The ARP cache on Metasploitable2 was examined again to confirm the legitimate MAC address of the gateway (10.16.113.196) before the attack. This serves as the baseline for comparison after the ARP spoofing attack is performed:

Command: `arp -n`



```
Metasploitable [Running] - Oracle VirtualBox
File   Machine   View   Input   Devices   Help

          collisions:0 txqueuelen:1000
          RX bytes:4645 (4.5 KB)  TX bytes:7070 (6.9 KB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:91 errors:0 dropped:0 overruns:0 frame:0
          TX packets:91 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:19301 (18.8 KB)  TX bytes:19301 (18.8 KB)

msfadmin@metasploitable:~$ arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
10.16.113.196            ether   22:50:BF:91:82:A4   C                     eth0
msfadmin@metasploitable:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
10.16.113.0     0.0.0.0         255.255.255.0   U     0      0        0 eth0
0.0.0.0         10.16.113.196   0.0.0.0         UG    100    0        0 eth0
msfadmin@metasploitable:~$ arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
10.16.113.196            ether   22:50:BF:91:82:A4   C                     eth0
msfadmin@metasploitable:~$
Right Ctrl
```

Figure 4: ARP table showing gateway 10.16.113.196 with real MAC 22:50:BF:91:82:A4

8. Step 5 — Run setup_lab.py (Pipeline Stage 1)

The first pipeline script setup_lab.py was executed on Kali Linux. This script installs Scapy, creates the logs directory, and enables IP forwarding so traffic continues to flow during the MITM attack. The script prints the roll number [03] and timestamp on every run:

Command on Kali:

```
cd ~/SKCT_03_ARPSpoofingTool/code
```

```
sudo python3 setup_lab.py
```

Figure 5: setup_lab.py output — Scapy installed, logs directory created, IP forwarding enabled

9. Step 6 — Run run_tool.py (Pipeline Stage 2 — TC-2)

The second pipeline script run_tool.py was executed. This script calls tool_main.py with the target IP (10.16.113.196) and gateway IP (10.16.113.196) and runs the ARP spoof for 30 seconds. The tool sent 28 bidirectional ARP packets poisoning both the victim and gateway ARP caches, successfully establishing a Man-in-the-Middle position:

Command on Kali:

```
sudo python3 run_tool.py
```

Figure 6: run_tool.py — 28 ARP spoof packets sent; analyze_results.py shows Roll No: 03, Timestamp, analysis complete

10. Step 7 — Test Case 3: Error Handling (Invalid IP)

To test error handling, the tool was run with an unreachable IP address (192.168.56.250) that does not exist on the network. The MAC resolution failed because Scapy's srp() returned an empty list, causing an

IndexError when accessing ans[0][1].hwsrc. The tool caught this cleanly and printed a descriptive error message with the roll number:

Command on Kali (TC-3):

```
sudo python3 tool_main.py 192.168.56.250 192.168.56.1
```

Figure 7: TC-3 error output — [03] ERROR: Could not resolve MAC address. Exception: list index out of range

11. Step 8 — Run analyze_results.py (Pipeline Stage 3)

The third pipeline script analyze_results.py was executed to parse the log file generated by run_tool.py and produce a structured analysis report. The script reads logs/run_output.txt, counts packets sent, identifies any errors, and saves the summary to logs/analysis_report.txt:

Command on Kali:

```
sudo python3 analyze_results.py
```

Figure 8: analyze_results.py — Analysis Report with Roll No: 03, Timestamp, pipeline complete

12. Output Summary

The ARP Spoofing Tool successfully demonstrated all three test cases. The pipeline executed all three stages (setup → run → analyze) in sequence. The tool poisoned the ARP cache on both the victim and gateway, intercepted bidirectional traffic, and handled error cases gracefully with descriptive messages.

Test Case	Target IP	Action	Result
TC-1: ARP Poison	10.16.113.57	One-way ARP poison — victim ARP table	Gateway MAC replaced with Kali MAC
TC-2: MITM (30s)	10.16.113.196	Bidirectional spoof — 28 packets sent	Traffic intercepted; run_tool.py complete
TC-3: Error Test	192.168.56.250	Invalid/unreachable IP resolution	IndexError caught — error logged cleanly

13. Verification

The successful execution of all three pipeline stages confirms that the ARP Spoofing Tool is working correctly. The tool was verified by observing ARP table changes on Metasploitable2, checking packet counts in logs/run_output.txt, and confirming graceful error handling in TC-3. All scripts include [03] roll number prefix and timestamps as required by the assignment.